Agentic AI App Builder Project

Project Introduction

The **Agentic AI App Builder Project** focuses on designing and developing an intelligent system capable of autonomously creating, managing, and enhancing applications using advanced AI techniques. Unlike traditional software development approaches that rely heavily on manual intervention, this project leverages **agent-based artificial intelligence** to automate decision-making, workflow execution, and application generation.

At its core, the system utilizes **Agentic AI**, where multiple intelligent agents collaborate to perform tasks such as requirement understanding, code generation, data retrieval, and iterative improvement. These agents are designed to operate with a degree of autonomy, enabling them to plan actions, execute tasks, and adapt based on feedback or changing requirements and the input prompts by the developer or user..

The project integrates modern AI concepts such as **Large Language Models (LLMs)**, **Retrieval-Augmented Generation (RAG)**, and **tool-based execution frameworks** to enhance the system's capability. This allows the application builder to not only generate code but also validate, refine, and optimize outputs in real time. Additionally, the system supports modularity, making it scalable and adaptable for different use cases.

The primary objective of this project is to demonstrate how **AI-driven automation** can significantly reduce development time, improve efficiency, and enable non-experts to build functional applications. By combining intelligence, autonomy, and adaptability, the Agentic AI App Builder represents a step toward the future of software development, where AI systems act as collaborative partners rather than just tools.

Overall, this project showcases the practical implementation of agentic workflows in building intelligent systems and highlights the transformative potential of AI in modern application development.

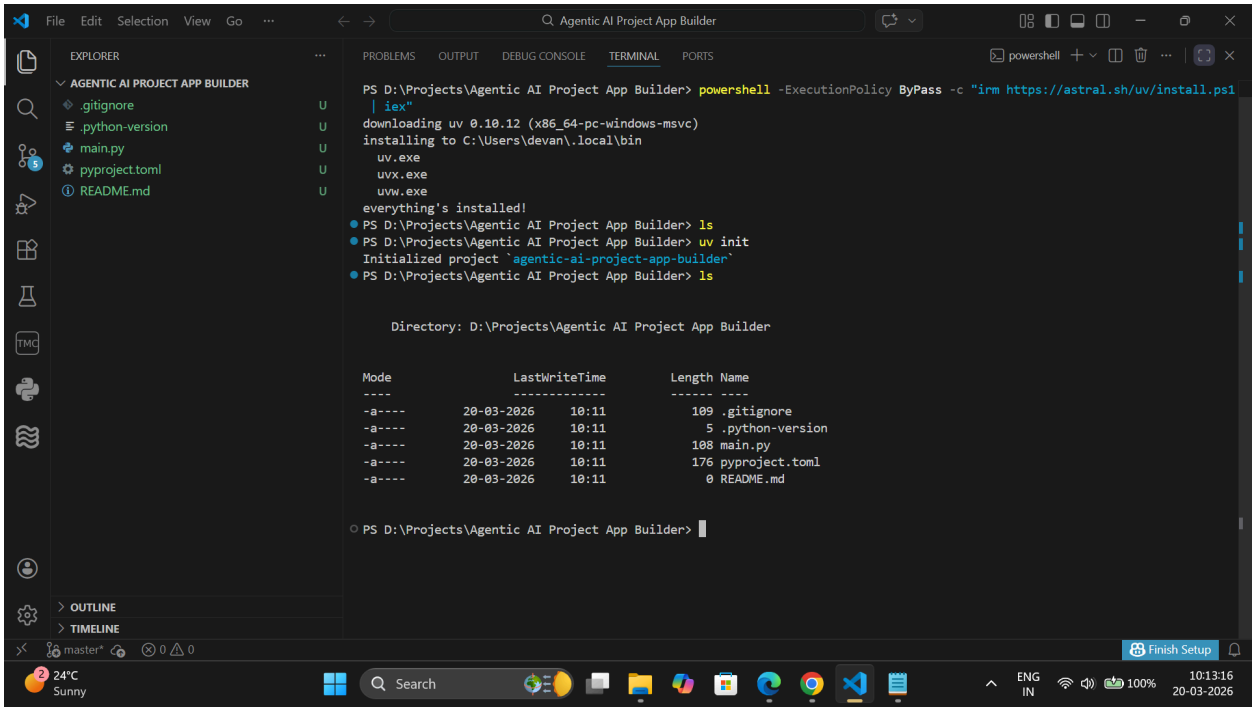
Project Implementation -

- so currently what I have done is - I created a folder in local d drive and opened it in vscode that folder. I open terminal windows powershell in vscode and ran this command - powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"

- then in the vs code powershell itself I run ls to check the list of files but since the project folder is empty so nothing gets printed.

- Then I run the command uv init

After running this command, the following files were created automatically with an untracked git status in green highlight color.



- even though the files are visible on the left, I ran again ‘ls’ command to officially check the status and presence of files in the project folder even.

- In the terminal in vs code there is a dropdown list called “Launch Profile…” to the right side of “New Terminal”, in that I select “git bash” option (currently I being on “powershell” yet).

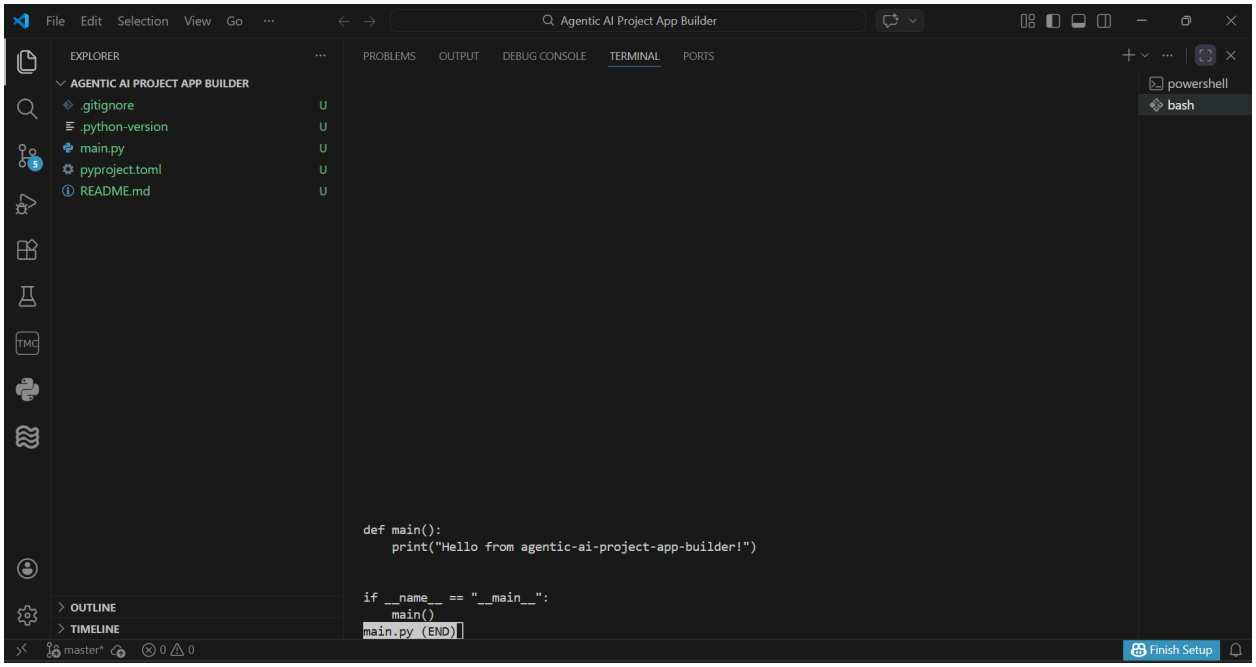
- this I do in order to be able to run the command ‘less main.py’ , this git bash has helped. So, further, it went forward to display -

=> **What less main.py does**

It opens the file main.py inside the terminal for viewing.

- You can **read the file content**
- You can **scroll up/down**
- You **cannot edit** (read-only)

-



- Then in order to check the file contents of pyproject.toml in a read only format only. I had to run the command ‘less pyproject.toml’.

- In git bash in the terminal of vs code the dollar symbol ‘$’ comes by default automatically so I just had to type and press enter to run the relevant git bash command simply thereby thus.

- However to first exit from this result display screen of git bash in terminal I had simply press ‘q’ to get out of this screen and back to git bash command running screen.

- project.toml is sort of like requirements.txt

- Added dependencies in pyproject.toml

- then I run ‘uv sync’. It will install all dependencies which I mentioned in project.toml. Also it creates a virtual environment too.

- Once venv is created, now its time to select the correct interpretor. Always ensure to select python interpretor with the local path => .\.venv\Scripts\python.exe

- After that then just go ahead and I ran the main.py file using the venv based python interpretor and I got the respective output.

- I created a .env file and in that I have written my groq api key.

- in main.py i am going to get, the user input about what kind of app they want to create.

- I created agent folder and in that I added agent.py in which I created graph.py. It named so because langraph is used and all the llm model code exists here.

- In the graph.py code, I proceed to go ahead and firstly try to print the response from llm after giving a prompt as an input.

- so for that I import ChatGroq from langchain_groq, then I save ChatGroq with model "`openai/gpt-oss-120b`"

- then I invoke the prompt using llm model variable, and then print the content received in return from the llm.

- However, then I proceeded further to call api_key using dotenv.

- Now the response the llm gives for prompt is a continuous text. But I wanted the response in a structured format following a particular schema. So, for that I firstly edited the code line llm.invoke to llm.with_structured_output(Schema).invoke. Then now while printing, resp.content is modified to resp only in print() function. In order to specify the schema I had to create a class called Schema which is derived class from the BaseModel class which belongs to the pydantic module. So thus, BaseModel is imported from pydantic module and that line of code I had to add too.

- So now actually since my aim is to create an app builder therefore,

further points ahead -

- Then I refactored the project structure to improve modularity and readability.
- I moved all prompt-related code from graph.py into a separate prompts.py file.
- I extracted all state definitions and Pydantic schemas into a dedicated states.py file.
- I updated graph.py to import prompts and state classes from these new modules.
- This helped in separating concerns between agent logic, prompts, and state handling.
- As a result, the codebase became cleaner and easier to extend for future agents like architect and coder.

- The agent should be multi modal, so, for that, added a function architect_prompt in prompt.py, which takes in a file as input and then builds around it too.

- Then I defined a new Pydantic class ImplementationTask to represent each individual coding task.
- This class contains filepath to specify which file needs to be modified and task_description to describe the implementation details.
- I also created another class TaskPlan which contains a list of ImplementationTask objects.

- This structure represents the detailed implementation steps generated by the architect agent.
- Additionally, I enabled extra fields in TaskPlan using model_config = ConfigDict(extra="allow") to allow flexibility in LLM responses.
- This helps in maintaining structured yet flexible output for downstream processing by the coder agent.

Challenges encountered -

1) **Faced warning of LF will be replaced by CRLF warning when doing first time git add . after git init -**

   So, solved it by running the command “git config --global core.autocrlf input”. further I created a .gitattributes file in my repository and in that I stated and specified the following command -

   * text=auto eol=lf

   Then finally I proceeded to renormalize the files while adding all the files and then doing git commit.

   git add --renormalize .

   And then git commit and git push.

2) **In graph.py initially I ran the line of code - from langchain import ChatGroq, but, `ChatGroq` is NOT inside `langchain` anymore -**

   LangChain has split integrations into separate packages.

   So, in git bash terminal in Pycharm, I had to run uv add langchain-groq, to install correct package

   then i changed the line of code to - from langchain_groq import ChatGroq

3) **ON just running this much code to print response to the prompt entered in LLM -**

```
from langchain_groq import ChatGroq

llm = ChatGroq(model = "openai/gpt-oss-1200")

resp = llm.invoke("Who invented kriya yoga. answer in 1 sentence")

print(resp.content)
```

=> It gave an error “groq.GroqError: The api_key client option must be set either by passing api_key to the client or by setting the GROQ_API_KEY environment variable”

So, I resolved that simply by adding the following lines of code -

```
from dotenv import load_dotenv

load_dotenv()
```

***By these the api key saved in .env file was successfully invoked, and got the output response successfully =>

Added `graph.py` for basic LLM interaction using ChatGroq.

Accepts a simple prompt and returns a single continuous text response.

Serves as the initial foundation for agent/graph-based workflow.

4) **Another Challenge encountered -**

fix: resolve Python version incompatibility and Pydantic warning by pinning project to Python 3.11

- Identified compatibility warning from LangChain/Pydantic:

  "Core Pydantic V1 functionality isn't compatible with Python 3.14"

- Root cause: project virtual environment was using Python 3.14, which is not fully supported by current LangChain dependencies

- Diagnosed version conflict:

  - `pyproject.toml` specified Python 3.11 requirement

  - `.python-version` was pinned to Python 3.14

  - uv prioritized `.python-version`, leading to incorrect interpreter selection

- Applied fixes:

  - Updated `.python-version` using `uv python pin 3.11`

  - Removed existing `.venv` tied to Python 3.14

  - Recreated virtual environment with Python 3.11

  - Reinstalled dependencies using `uv sync`

- Verified fix:
  - Confirmed interpreter version via `uv run python --version`
  - Warning related to Pydantic V1 no longer appears
  - LLM pipeline (ChatGroq + LangChain) functioning correctly

- Improved configuration:
  - Adjusted Python requirement strategy for better compatibility and stability
  - Ensured alignment between environment pinning and dependency constraints

This ensures a stable, reproducible environment aligned with current AI ecosystem support (LangChain, FastAPI, etc.).

5) **when running the code lines -**

set_debug(True)

set_verbose(True)

the module used to import it are -

from langchain.globals import set_verbose, set_debug

but, `langchain.globals` no longer exists

so error or issue solved by using and running -

from langchain_core.globals import set_verbose, set_debug

6) **During executing the file graph.py after coding agent included, I faced this error -**

Challenge: ModuleNotFoundError for local modules

Encountered import error:

ModuleNotFoundError: No module named 'prompts'

Root Cause:

Python was unable to locate local modules due to incorrect import paths when running the script via uv. The execution context did not include the project root in PYTHONPATH.

Solution:

Replaced relative/local imports with absolute imports:

from agent.prompts import *

from agent.states import *

Key Learning:

- Always prefer absolute imports in multi-module projects
- Execution context (uv, IDE, CLI) affects module resolution
- Proper project structuring avoids import-related issues

Status:

✅ Issue resolved and verified using PyCharm debugger